KV-Cache Eviction vs Quantisation

Code documentation for `kv_eviction_vs_quant.py` · Sukrit Gupta · 1 November 2026

What the script does. On one small open model, it compares two ways of shrinking the KV cache, dropping tokens (H2O-style eviction) and storing them in fewer bits (quantisation), plus both together. It runs three long-context tasks and records accuracy, latency, GPU energy, peak memory and KV-cache size for every answer.

Status. Written and syntax-checked. The scoring and summary code was tested on dummy data, and the library interfaces were checked against current documentation. **It has not yet been run on a GPU.**

1. Research question and design

Question. For long inputs, how do H2O-style KV-cache eviction (Zhang et al., NeurIPS 2023; Beidi Chen, senior author) and KV-cache quantisation compare, do they combine, and does the answer change by task? Costs are measured the way Fernandez et al. (ACL 2025, with Emma Strubell) recommend: real latency and energy on the GPU, not FLOP estimates.

Conditions

Condition	Tokens evicted	KV bits	KV cache, whole model (4k tokens)	What it tests
full_fp16	0%	16	117.4 MB	Baseline, exact
h2o_50	50%	16	58.7 MB	Drop half the tokens
h2o_75	75%	16	29.4 MB	Drop three quarters
kv_int4	0%	4	29.4 MB	Keep all tokens, 4-bit
kv_int2	0%	2	14.7 MB	Keep all tokens, 2-bit
h2o_50+kv_int4	50%	4	14.7 MB	Both together

Two equal-memory pairs make the cleanest comparison: h2o_75 vs kv_int4 (both 29.4 MB) and kv_int2 vs h2o_50+kv_int4 (both 14.7 MB). At the same memory, is it better to drop tokens or to lower precision?

Tasks (RULER, 4,096-token contexts)

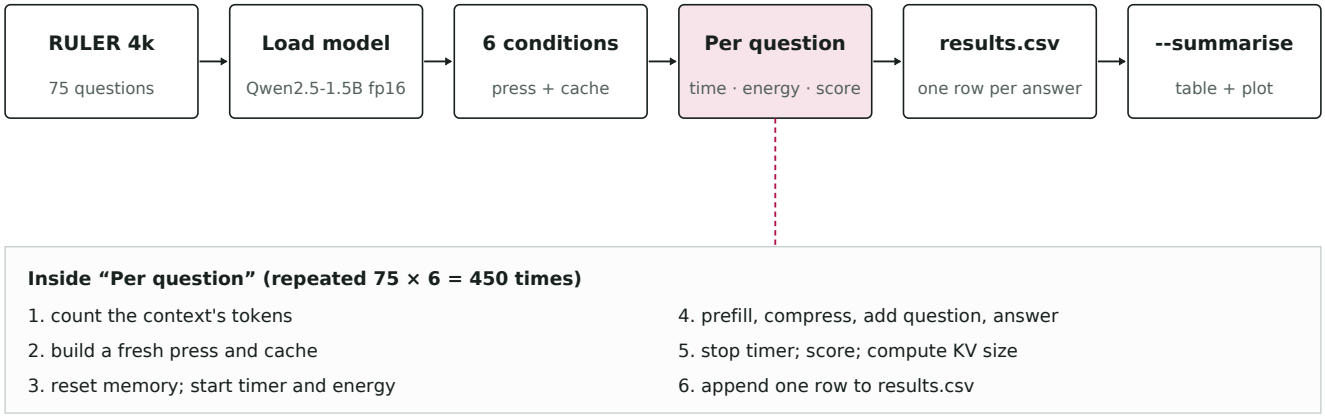
Task	What it tests	Example	Scoring	Max tokens
niah_single_1	Retrieve one hidden fact	“What is the special magic number for wandering-age?” → 8090293	Fraction of references found	128
vt	Follow chained assignments (multi-hop)	“Find all variables assigned 15311” → 5 names	Fraction found (20 points each)	30
qa_1	Answer from real documents with distractors	“In what country is Normandy located?” → France	Full credit if any reference appears	32

Sample. 25 questions per task, chosen with a fixed seed, so every condition answers the same 75 questions: 6 × 75 = **450 answers**, about 1.5–2.5 hours on a free T4 GPU.

Model

Property	Qwen2.5-1.5B-Instruct
Layers	28
Query heads / KV heads	12 / 2 (grouped-query attention, 6 query heads per KV head)
Head size	128
Weights	≈1.54B parameters, ≈3.1 GB in fp16
KV cache	2 × 28 × 2 × 128 × 2 bytes = 28.7 KB per token (1 KB per token per layer)
Run as	fp16 with eager attention (needed by the H2O-style press), on CUDA

2. Code architecture



Component	Role
CONDITIONS	The experiment grid: each condition is an eviction ratio (0, 0.5, 0.75) plus KV bits (16, 4, 2).
EnergyMeter	Context manager that reads GPU energy in joules from NVIDIA's NVML energy counter, or integrates sampled power if the counter is unavailable. Reports NaN without NVML.
score_example	RULER's exact-answer metric: qa tasks get 100 if any reference appears; other tasks get the share of references found × 100. Case-insensitive substring match.
load_model	Loads the model in fp16 with eager attention on the GPU (H2O-style scoring needs the attention weights, which faster kernels never store).
make_press	Builds kvpress's ObservedAttentionPress (H2O-style eviction) for a given ratio, or None.
make_cache	Builds a fresh transformers QuantizedCache (quanto backend, 4 or 2 bits), handling both the old and new transformers APIs; None means the default full-precision cache.
kv_bytes_per_token	Computes KV bytes per token from the model config (used for the analytic KV size).
run	Loads data and model, loops over conditions and questions, measures, scores and appends each answer to results.csv. Resumes from an existing results.csv.
summarise	Builds summary.csv (accuracy per task and condition, plus mean latency, energy, peak memory and KV size) and accuracy_vs_kv.png (one panel per task).
__main__	Command-line arguments; runs run() or summarise().

3. Running the experiment

Setup

- Start a free Kaggle (or Colab) notebook with a T4 GPU. An NVIDIA GPU is required: NVML energy readings and the quanto KV cache need CUDA.

- Install and run:

```
pip install -q kvpress optimum-quanto nvidia-ml-py datasets pandas matplotlib
python kv_eviction_vs_quant.py --n 2          # smoke test, 6 answers per condition
python kv_eviction_vs_quant.py --n 25         # full run, 450 answers
python kv_eviction_vs_quant.py --summarise    # summary.csv + accuracy_vs_kv.png
```

Arguments

Flag	Default	Meaning
--model	Qwen/Qwen2.5-1.5B-Instruct	Any causal LM kvpress supports
--config	4096	RULER context length: 4096, 8192 or 16384
--tasks	niah_single_1 vt qa_1	Any of RULER's 13 task names
--n	25	Questions per task
--conditions	all six	Run a subset, e.g. --conditions full_fp16 h2o_50
--out	results.csv	Results file (appended to, so runs resume)
--summarise	off	Build the summary table and plot from results.csv

What happens for each question

Step	Action	Where
1	Count the context's tokens (~4,096)	tok(row['context'])
2	Build a fresh press and cache for this condition, so nothing leaks between questions	make_press, make_cache
3	Reset the peak-memory counter, start the timer, read the energy counter	torch.cuda, EnergyMeter
4	Prefill: the model reads the context and fills the KV cache layer by layer	pipe(...)
5	Compress: the press deletes low-attention tokens and/or the cache stores values in 4 or 2 bits	press, cache
6	Add the question, then generate greedily up to max_new_tokens	pipe(...)
7	Stop the timer and energy reading; record peak memory	EnergyMeter
8	Score the answer; compute KV size = tokens × (1 − ratio) × 28.7 KB × bits ÷ 16	score_example
9	Append one row to results.csv	pandas

Outputs

File	Contents
results.csv	One row per answer: condition, row_id, task, score, latency_s, energy_j, peak_mem_gb, ctx_tokens, kv_ctx_mb, pred (first 200 characters), refs
summary.csv	Accuracy per task × condition, plus mean KV size, latency and energy, max peak memory, and n
accuracy_vs_kv.png	Accuracy against KV size, one panel per task, every condition labelled

4. Key mechanisms

4.1 The KV cache

For each layer, keys and values each have shape `(batch, KV heads, tokens, head size)` = `(1, 2, T, 128)`. Every token adds one row of 128 numbers per KV head. At 4,096 tokens that is 1 KB × 4,096 = 4.19 MB per layer for keys and values together, or 117 MB across 28 layers. The model's weights (≈3.1 GB) are loaded once; the cache is built fresh for every question and discarded afterwards.

4.2 H2O-style eviction (kvpress ObservedAttentionPress)

Runs once per layer, after the context has been read and **before the question arrives**:

Step	Plain language	Code	Shape
1	Eager attention gives each token's attention to every earlier token (rows = reading token)	attentions	(1, 12, T, T)
2	Add up the attention each token received	attentions.sum(2)	(1, 12, T)
3	Divide by how many tokens could see it, giving average attention	÷ arange(T, 0, −1)	(1, 12, T)
4	Average the 6 query heads that share each KV head	.view(1, 2, 6, T).mean(2)	(1, 2, T)
5	Keep the top int(T × (1 − ratio)) tokens per KV head	scores.topk(n_kept)	indices
6	Delete the other rows of K and V	keys.gather(2, idx)	(1, 2, T′, 128)

Differences from the original H2O: H2O evicts continuously during generation, keeps a window of recent tokens, and has seen the question. This press compresses once, has no recent window, and scores the context before the question exists, which is why a hidden fact can be deleted on needle tasks. Report it as “H2O-style”.

4.3 KV quantisation (transformers QuantizedCache)

Keeps every token but stores keys and values in 4 or 2 bits using the quanto backend. The most recent tokens (128 by default) stay in full precision until the buffer fills; values are converted back to fp16 whenever attention reads them, which adds work at every step.

4.4 Measurement

- **Accuracy:** RULER exact-answer match (Section 2). Deterministic, because kvpress decodes greedily.
- **Latency:** wall-clock time for the whole answer (prefill + decoding), with GPU synchronisation before and after.
- **Energy:** joules per answer from the NVML energy counter.
- **Peak memory:** torch.cuda.max_memory_allocated per answer.
- **KV size:** calculated from the config, ignoring quantisation scale overhead (Section 6).

5. What to expect and how to read the results

- **Lead with the equal-memory pairs.** They answer “drop tokens or lower precision?” at a fixed budget.
- **Look for task differences.** Eviction should hurt niah and vt (one lost token breaks the answer) more than qa_1 (real text with redundancy).
- **Expect little latency gain at this scale.** Generating a token mostly means reading weights plus cache; at 4k tokens the cache is tiny next to the weights. Prefill is not faster, and quantised conditions add dequantisation work.

Situation (T4, ~320 GB/s)	Weights read	KV read	Time per token	With 50% eviction
This experiment: 4k tokens, 1 request	3.1 GB	0.117 GB	≈10.1 ms	≈9.9 ms (≈2% faster)
32k tokens, 16 requests	3.1 GB	≈15 GB	≈57 ms	≈33 ms (≈41% faster)

Back-of-envelope estimates from time ≈ bytes read ÷ memory bandwidth; real timings also include compute and overheads.

- **Treat small differences as noise.** With 25 questions per task, each question is 4 points and the 95% margin of error is roughly ±20 points. Use paired comparisons, since every condition answers the same questions.

6. Limitations

Limitation	Effect	How to handle it
Not yet run on a GPU	Library versions may differ	Smoke test with --n 2 first
h2o_50+kv_int4 is undocumented in kvpress	May fail	The script skips a condition that fails its warm-up
H2O-style, not original H2O	One-shot, question-agnostic, no recent window	Name it accurately; see the extension below
Analytic KV size	Ignores quantisation scales and the fp16 buffer	State it in the README
Eager attention	Slower, more memory at long contexts (~6.4 GB per layer at 16k)	Keep to 4k or 8k on a T4
Small sample, one model, one length	Wide error bars; may not generalise	Report as a pilot; optionally add 8k or a 3B model

7. Planned extension: question-aware eviction (not yet in the code)

Hypothesis. The needle is lost because the press scores the context before the question exists. If the question is placed last and tokens are scored only by the question's attention, the needle should survive, even at 75% eviction. This is the idea behind SnapKV.

Why the question goes last, not first

Scores come from attention a token receives from tokens *after* it. A question placed first cannot look at the needle, so its effect is indirect. Placed last, the question's tokens look back over the whole document. The current press averages over all ~4,000 readers, which would drown out the ~30 question tokens, so the new press reads only the question's rows of the attention table.

```
@dataclass
class QuestionAwarePress(ObservedAttentionPress):
    n_question: int = 32   # final tokens that belong to the question

    def score(self, module, hidden_states, keys, values, attentions, kwargs):
        # rows read by the question tokens only, averaged -> (1, 12, T)
        q_att = attentions[:, :, -self.n_question:, :].mean(2)
        bsz, n_kv, n_tokens, _ = keys.shape
        # average the 6 query heads that share each KV head -> (1, 2, T)
        scores = q_att.view(bsz, n_kv, -1, n_tokens).mean(2)
        # never evict the question itself
        scores[..., -self.n_question:] = torch.finfo(scores.dtype).max
        return scores
```

New condition	Prompt layout	Press	Prediction for niah
h2o_50_qfirst	question, context, question	ObservedAttentionPress	Small improvement at most
qaware_50	context, question (compressed together)	QuestionAwarePress	Needle mostly survives
qaware_75	context, question (compressed together)	QuestionAwarePress	Needle mostly survives

Validation checks

- Measure `n_question` from the joined text: tokens(context + question) − tokens(context).
- Decode the last `n_question` tokens and confirm they are the question.
- Save the scores from a middle layer and decode the top-scoring tokens for one niah example; they should include the needle's key or number.

Trade-off to report

Question-aware compression must be redone for every question; question-agnostic compression can be done once per document and reused. The result becomes accuracy versus reusability. Estimated cost: about 45 minutes more GPU time.

Other small additions

- **Separate prefill and decoding times**, so cache savings show in decoding, where they happen.
- **H2OPress with a recent window** (keep the newest tokens plus heavy hitters), closer to the original H2O budget.
- **Needle tracking:** record whether the needle's tokens were kept in each layer, to explain failures.

8. References

- H2O: Heavy-Hitter Oracle (Zhang et al., 2023): https://arxiv.org/abs/2306.14048
- ShadowKV (Sun et al., ICML 2025): https://proceedings.mlr.press/v267/sun25b.html
- TriForce (Sun et al., 2024): https://arxiv.org/abs/2404.11912
- KIVI (Liu et al., ICML 2024): https://arxiv.org/abs/2402.02750
- Energy Considerations of LLM Inference (Fernandez et al., ACL 2025): https://preview.aclanthology.org/setup/2025.acl-long.1563
- RULER (Hsieh et al., 2024): https://arxiv.org/abs/2404.06654
- FlashAttention (Dao et al., 2022): https://arxiv.org/abs/2205.14135
- Qwen2.5 Technical Report: https://arxiv.org/abs/2412.15115
- kvpress (NVIDIA): https://github.com/NVIDIA/kvpress
- RULER data used (simonjegou/ruler): https://huggingface.co/datasets/simonjegou/ruler